API Produtivo: guia de integração para o front-end

Repositório do back-end: https://github.com/romeroflp/produtivoBackend

Como usar: se é a sua primeira vez consumindo uma API, leia as seções 1 a 4 em ordem. A seção 5 é a referência de cada rota, para consultar enquanto desenvolve.

Os formatos descritos aqui (rotas, corpos enviados e respostas) são os **definitivos**. Algumas rotas ainda estão sendo ajustadas no back-end, mas os contratos não vão mudar. Você pode implementar o front agora. Se uma rota devolver erro 500, avise o Felipe.

1. O que é esta API?

O Produtivo é um gerenciador de tarefas. Cada pessoa cria uma conta, faz login e organiza suas tarefas em categorias (Trabalho, Estudos, etc.).

O back-end não tem telas. Ele só expõe endereços, chamados de **endpoints**, que recebem e devolvem dados em formato **JSON**. O seu front-end desenha as telas e conversa com esses endpoints usando `fetch` (ou axios).

2. Conceitos básicos

2.1 Requisição e resposta

O front envia uma **requisição** e o back devolve uma **resposta**. Uma requisição tem quatro partes:

Parte	O que é	Exemplo
Método	O tipo de ação	GET, POST, PATCH, PUT, DELETE
URL	Qual recurso acessar	http://localhost:8080/api/task/myTasks
Headers	Informações extras	Content-Type, Authorization
Body	Dados enviados (POST, PUT, PATCH)	{"title": "Estudar"}

GET busca dados, POST cria algo ou envia dados, PATCH e PUT atualizam, DELETE apaga.

2.2 Status code

Toda resposta traz um número dizendo como foi:

- **2xx**: sucesso (200 OK, 201 Created, 204 sem conteúdo)
- **4xx**: problema na requisição (400 dados inválidos, 401 não autenticado, 403 sem permissão, 404 não encontrado, 409 conflito)
- **5xx**: erro dentro do servidor

Com `fetch`, um 400 ou 500 **não** gera exceção. Verifique sempre `response.ok` (verdadeiro para 200 a 299).

2.3 JSON

Texto com estrutura: objetos `{}` com pares chave/valor, listas `[]`, textos entre aspas duplas, números, `true` / `false` e `null`. Para enviar, use `JSON.stringify(objeto)`. Para ler a resposta, `response.json()`.

2.4 Autenticação com token (JWT)

Quase todas as rotas só funcionam para usuário logado:

1. Login ou cadastro: o back devolve um `token` (texto longo).
2. Você guarda o token (por exemplo, no `localStorage`).
3. Em toda requisição protegida, envia o header `Authorization: Bearer SEU_TOKEN` (a palavra Bearer, um espaço e o token).
4. O token vale cerca de 4 horas. Depois disso o usuário precisa logar de novo.

Sem token, ou com token inválido ou expirado, a resposta é 401 ou 403. Trate os dois do mesmo jeito: apague o token e leve o usuário para o login.

2.5 CORS

Por segurança, o navegador impede que uma página de um endereço (ex.: localhost:5173) chame uma API em outro (ex.: localhost:8080), a menos que a API autorize. Essa autorização se chama CORS. Se o console mostrar "blocked by CORS policy", o problema é a origem não autorizada, não o seu código (veja 3.3).

3. Configuração

3.1 URL base

Rodando localmente: `http://localhost:8080`. Todas as rotas começam com `/api`, por exemplo `http://localhost:8080/api/auth/login`. Guarde a URL base numa variável de ambiente (ex.: `VITE_API_URL`) para trocar quando o back for publicado.

3.2 Rodando o back-end

Precisa de Java 17 e de um PostgreSQL com um banco chamado `produtivodatabase`. As credenciais do banco e do serviço de e-mail ficam na configuração do back (peça os valores ao Felipe). As tabelas são criadas sozinhas ao iniciar. Comando, na raiz do projeto: `./mvnw spring-boot:run` (no Windows: `mvnw.cmd spring-boot:run`).

3.3 Origem liberada (CORS)

O back libera a origem `http://localhost:5173` (porta padrão do Vite), com os métodos GET, POST, PUT, PATCH, DELETE e OPTIONS e os headers `Authorization` e `Content-Type`. Se o seu front rodar em outra porta, mude a porta do servidor de desenvolvimento ou peça ao Felipe para liberar a origem. Não use `credentials: 'include'`, pois a API não usa cookies. Quando o front for publicado, o novo endereço também precisa ser liberado no back.

4. Código base para o front

4.1 Função auxiliar para chamar a API

Em vez de repetir `fetch` em toda tela, centralize numa função. Ela adiciona os headers, converte o corpo e trata respostas que não são JSON (204 sem corpo, texto simples) e os erros.

```
const API_URL = 'http://localhost:8080';

export class ApiError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}

export async function api(path, { method = 'GET', body, auth = true } = {}) {
  const headers = { 'Content-Type': 'application/json' };
  const token = localStorage.getItem('token');
  if (auth && token) headers['Authorization'] = `Bearer ${token}`;

  const response = await fetch(`${API_URL}${path}`, {
    method,
    headers,
    body: body ? JSON.stringify(body) : undefined,
  });
```

```
  // Lê como texto: funciona para JSON, texto simples e resposta vazia (204)
  const text = await response.text();
  let data = null;
  if (text) {
    try { data = JSON.parse(text); } catch { data = text; }
  }

  if (!response.ok) {
    // Sessão expirada ou token inválido (só vale para rotas protegidas)
    if (auth && (response.status === 401 || response.status === 403)) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    throw new ApiError(response.status, data?.message ?? 'Erro inesperado.
Tente novamente.');
  }
  return data;
}
```

Exemplo de uso:

```
const { token } = await api('/api/auth/login', {
  method: 'POST',
  auth: false,
  body: { email, password },
});
localStorage.setItem('token', token);

const categorias = await api('/api/categories');
```

4.2 Lendo e-mail e expiração do token

O login devolve só o token. O token carrega o e-mail (`sub`) e o momento de expiração (`exp` , em segundos). Para ler:

```
export function readToken(token) {
  const base64 = token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
  return JSON.parse(atob(base64)); // { iss, sub: 'email', exp: 1767225600 }
}
// expirou? readToken(token).exp * 1000 < Date.now()
```

Isso serve só para exibir dados e saber quando pedir novo login. Quem valida o token é sempre o back. O **nome** do usuário vem na resposta do cadastro; guarde-o nessa hora.

4.3 Fluxo geral do app

1. Cadastro ou login, e salvar o token.
2. Ao abrir o app: se há token e ele não expirou, mostrar as telas; senão, a tela de login.
3. Carregar as categorias (`GET /api/categories`) e as tarefas (`GET /api/task/myTasks`).
4. Qualquer 401 ou 403 numa rota protegida: apagar o token e voltar ao login.

5. Referência dos endpoints

5.1 Resumo

Método	Rota	Precisa de token	Para quê
POST	/api/auth/login	Não	Fazer login
POST	/api/auth/register	Não	Criar conta
POST	/api/mail/forgotPassword	Não	Enviar código de redefinição por e-mail
POST	/api/resetPassword/validate	Não	Conferir o código recebido
POST	/api/resetPassword/confirm	Não	Definir a nova senha
GET	/api/categories	Sim	Listar categorias
POST	/api/categories/save	Sim	Criar categoria
PATCH	/api/categories/updateCategory	Sim	Editar categoria
DELETE	/api/categories/{id}	Sim	Apagar categoria
POST	/api/task/save	Sim	Criar tarefa
GET	/api/task/myTasks	Sim	Listar minhas tarefas (paginado)
GET	/api/task/{id}	Sim	Buscar tarefa por id
GET	/api/task/search/{title}	Sim	Buscar por título
GET	/api/task/fillTasks	Sim	Filtrar tarefas
PATCH	/api/task/updateTask	Sim	Editar tarefa
DELETE	/api/task/delete/{id}	Sim	Apagar tarefa

Método	Rota	Precisa de token	Para quê
PUT	/api/users/updateName	Sim	Trocar o nome
DELETE	/api/users/deleteUser	Sim	Apagar a própria conta

5.2 Autenticação

POST /api/auth/login

Corpo:

```
{ "email": "maria@email.com", "password": "minhaSenha123" }
```

Resposta 200:

```
{ "token": "eyJhbGciOiJIUzI1NiIs..." }
```

Se o e-mail ou a senha estiverem errados, a resposta é 401 com a mensagem "Dados inválidos. Verifique se o email e senha foram digitados corretamente.". Aqui o 401 significa credencial errada, não sessão expirada. Por isso o login é chamado com `auth: false` na função auxiliar.

POST /api/auth/register

Corpo:

```
{ "name": "Maria Silva", "email": "maria@email.com", "password":
"minhaSenha123" }
```

Campo	Regra
name	Obrigatório, de 3 a 30 caracteres
email	Obrigatório, formato de e-mail válido, único no sistema
password	Obrigatório, de 8 a 12 caracteres

Valide essas regras no front antes de enviar.

Resposta 201 (já traz o token, não precisa fazer login em seguida):

```
{
  "id": "0d4f6c1e-8a53-4c0e-9d7a-2b1f5a9c3e11",
  "name": "Maria Silva",
  "email": "maria@email.com",
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Se o e-mail já estiver cadastrado, a resposta é 409.

5.3 Recuperação de senha

São 3 chamadas em sequência. Nenhuma exige token.

Passo 1: POST /api/mail/forgotPassword

```
{ "name": "Maria Silva", "recipientEmail": "maria@email.com" }
```

Resposta 200:

```
{
  "message": "Email enviado para Maria Silva no email: maria@email.com",
  "messageId": "<id-do-provedor>",
  "token": "7b2e9c1a-5d44-4f0b-8e6a-1c3d9f2a7b10"
}
```

O usuário recebe por e-mail um código de 5 dígitos. **Guarde o campo `token`** no estado do front: ele é necessário no passo 3 e vale por 5 minutos. O código de 5 dígitos **não** vem na resposta; o usuário deve digitá-lo na tela.

Passo 2: POST /api/resetPassword/validate

```
{ "codeProvideByUser": "48213" }
```

Resposta 200: o JSON puro `true` ou `false` (sem objeto em volta). Se `false`, mostre "código incorreto" e deixe o usuário tentar de novo.

Passo 3: POST /api/resetPassword/confirm

```
{
  "resetToken": "7b2e9c1a-5d44-4f0b-8e6a-1c3d9f2a7b10",
  "newPassword": "novaSenha123",
  "confirmNewPassword": "novaSenha123"
}
```

Resposta 200: **texto simples**, não JSON: `Nova senha atualizada com sucesso!` (a função auxiliar da seção 4.1 já lida com isso).

A senha nova deve ter de 8 a 12 caracteres, e `newPassword` e `confirmNewPassword` devem ser iguais. O passo 2 precisa ter sido concluído com sucesso antes do passo 3.

5.4 Categorias

Toda tarefa pertence a uma categoria. Há duas origens:

- **Categorias padrão**: 20 categorias do sistema, iguais para todos, com `userId: null`.
- **Categorias do usuário**: criadas por ele, com `userId` preenchido.

GET /api/categories

Lista as padrão mais as do usuário logado.

```
[
  { "id": 1, "name": "TRABALHO", "userId": null, "colorIcon": "#1E88E5" },
  { "id": 21, "name": "Faculdade", "userId": "0d4f6c1e-8a53-...",
"colorIcon": "#112233" }
]
```

`colorIcon` é uma cor hexadecimal para colorir o ícone ou a etiqueta. Categorias padrão: TRABALHO, ESTUDOS, PESSOAL, SAÚDE, EXERCÍCIOS, COMPRAS, FINANÇAS, CASA, FAMÍLIA, LAZER, HOBBIES, PROJETOS, REUNIÕES, VIAGENS, EVENTOS, METAS, ROTINA, URGENTE, IMPORTANTES, OUTROS. Nunca fixe os `id` no código: use sempre os que a API devolve.

Todas as categorias da lista, padrão ou do usuário, podem ser usadas ao criar tarefas. Só as do usuário (`userId` preenchido) podem ser editadas ou apagadas, então esconda esses botões nas categorias padrão (`userId === null`).

POST /api/categories/save

```
{ "name": "Faculdade", "colorIcon": "#112233" }
```

Resposta 201: a categoria criada (`id`, `name`, `userId`, `colorIcon`). `colorIcon` deve ser hexadecimal no formato `#RGB` ou `#RRGGBB`.

PATCH /api/categories/updateCategory

```
{ "id": 21, "name": "Faculdade 2026", "colorIcon": "#223344" }
```

Resposta 200: a categoria atualizada. Envie sempre `id`, `name` e `colorIcon`.

DELETE /api/categories/{id}

Exemplo: `DELETE /api/categories/21`. Resposta **204**, sem corpo. Só é possível apagar uma categoria que não tenha tarefas (se tiver, a resposta é 409). Peça confirmação ao usuário antes de chamar.

5.5 Tarefas

Todas as rotas desta seção exigem token e só enxergam as tarefas do usuário logado.

POST /api/task/save

```
{
  "title": "Estudar Spring Security",
  "description": "Ler o capítulo sobre JWT",
  "categoryId": 21,
  "statusTask": "PENDING",
  "expirationDate": "2026-11-05T18:30:00",
  "recurrent": false
}
```

Campo	Obrigatório	Observação
title	Sim	Não pode ser vazio
description	Não	Texto livre
categoryId	Sim	Id de qualquer categoria devolvida por `GET /api/categories`
statusTask	Sim	Um dos valores da seção 6.1 (para tarefa nova, use `PENDING`)
expirationDate	Não	Data e hora de vencimento (formato na seção 6.2)
recurrent	Sim	`true` ou `false`

Resposta 201:

```
{
  "id": 15,
  "title": "Estudar Spring Security",
  "description": "Ler o capítulo sobre JWT",
  "categoryId": 21,
  "statusTask": "PENDING",
  "createdIn": "2026-10-26T14:03:21.123456",
  "expirationDate": "2026-11-05T18:30:00",
  "recurrent": false
}
```

GET /api/task/myTasks

Lista as tarefas do usuário, **paginadas**. Parâmetros opcionais na URL:

Parâmetro	Significado	Exemplo
page	Número da página, começando em 0	page=0
size	Itens por página (padrão 20)	size=10
sort	Campo e direção	sort=createdIn,desc

Exemplo: `GET /api/task/myTasks?page=0&size=10&sort=expirationDate,asc`. Campos aceitos em `sort`: `createdIn`, `expirationDate`, `title`, `statusTask`, `recurrent`.

Resposta 200:

```
{
  "content": [
    {
      "taskId": 15,
      "userId": "0d4f6c1e-8a53-...",
      "title": "Estudar Spring Security",
      "description": "Ler o capítulo sobre JWT",
      "createdIn": "2026-10-26T14:03:21.123456",
      "expirationDate": "2026-11-05T18:30:00",
      "categoryId": 21,
      "name": "Faculdade",
      "statusTask": "PENDING",
      "recurrent": false,
      "colorIcon": "#112233"
    }
  ],
  "totalElements": 1,
  "totalPages": 1,
```

```
  "number": 0,
  "size": 10,
  "first": true,
  "last": true
}
```

Use `content` para a lista e `totalPages` / `last` para a paginação. A resposta de página tem outros campos (como `pageable` e `sort`); pode ignorá-los.

Não confunda os formatos: nesta rota o id da tarefa se chama `taskId` e `name` é o **nome da categoria** (`colorIcon` é a cor dela). Nas demais rotas de tarefa o id se chama `id` e só vem `categoryId`.

GET /api/task/{id}

Busca uma tarefa pelo id. Exemplo: `GET /api/task/15`. Resposta 200, no mesmo formato do `POST /api/task/save`.

GET /api/task/search/{title}

Busca tarefas cujo título **contém** o texto, sem diferenciar maiúsculas de minúsculas. Exemplo: `/api/task/search/spring`. Se o texto tiver espaço ou acento, use `encodeURIComponent(texto)`. Resposta 200: lista de tarefas no formato do `POST /api/task/save`. Quando nada é encontrado, a resposta é uma lista vazia `[]`.

GET /api/task/fillTasks

Filtra tarefas por parâmetros opcionais, que podem ser combinados:

Parâmetro	Exemplo	Efeito
title	title=spring	Título contém o texto (sem diferenciar maiúsculas)
statusTask	statusTask=PENDING	Só desse status
recurrent	recurrent=true	Só recorrentes (true) ou não (false)
startDate	startDate=2026-10-26	Criadas nesse dia; com `endDate`, criadas a partir dessa data
endDate	endDate=2026-10-31	Criadas até essa data, inclusive (use junto com `startDate`)

Datas no formato `aaaa-mm-dd` e filtram pela **data de criação** da tarefa, não pelo vencimento. Exemplo: `/api/task/fillTasks?statusTask=PENDING&recurrent=false`. Resposta 200: lista de tarefas no formato do `POST /api/task/save`.

PATCH /api/task/updateTask

Mesmo corpo da criação, **com o campo `id`** da tarefa:

```
{
  "id": 15,
  "title": "Estudar Spring Security",
  "description": "Ler o capítulo sobre JWT e OAuth",
  "categoryId": 21,
  "statusTask": "IN_PROGRESS",
  "expirationDate": "2026-11-05T18:30:00",
  "recurrent": false
}
```

Resposta 200: a tarefa atualizada (formato do `save`). O campo `createdIn` não é alterado.

Envie sempre **todos** os campos, partindo dos valores atuais da tarefa. Para marcar como concluída, mande o objeto inteiro com `"statusTask": "COMPLETED"`.

DELETE /api/task/delete/{id}

Exemplo: `DELETE /api/task/delete/15`. Resposta **204**, sem corpo. Peça confirmação ao usuário antes de chamar, pois não há como desfazer.

5.6 Conta do usuário

PUT /api/users/updateName

```
{ "newName": "Maria Souza" }
```

O nome deve ter de 3 a 30 caracteres, com letras (acentuadas ou não), espaços, hífen ou apóstrofo. Resposta 200:

```
{ "name": "Maria Souza", "email": "maria@email.com" }
```

DELETE /api/users/deleteUser

Apaga a conta de quem está logado, **com todas as tarefas e categorias dele**. Resposta 204, sem corpo. Peça confirmação clara e, em seguida, apague o token e volte ao login. Não há como desfazer.

6. Valores válidos

6.1 statusTask

O back envia e recebe o **nome em inglês**. Os rótulos em português são sugestão de exibição, e a conversão é feita no front.

Valor na API	Sugestão de exibição
PENDING	Pendente
IN_PROGRESS	Em progresso
COMPLETED	Completo
OVERDUE	Atrasado
CANCELED	Cancelado

O back não muda o status sozinho: quem decide é o front. Para exibir "atrasado", compare `expirationDate` com a data atual, ou envie `OVERDUE` via `updateTask`.

6.2 Datas e horas

Datas de tarefa são texto no padrão ISO, **sem fuso horário**: `2026-11-05T18:30:00`. O valor de um `<input type="datetime-local">` (ex.: `2026-11-05T18:30`) pode ser enviado direto. Para exibir: `new Date(valor).toLocaleString('pt-BR')`. O campo `createdIn` vem com frações de segundo; o `new Date()` aceita normalmente.

7. Respostas de falha

Quando uma chamada falha, o status code diz o motivo e o corpo traz o campo `message` com um texto pronto para exibir. A função auxiliar da seção 4.1 já lê esse campo.

Status	Quando acontece
400	Dados enviados inválidos ou código/senhas de redefinição incorretos
401	Login com e-mail ou senha errados; token ausente ou expirado
403	Token inválido ou sessão expirada
404	Tarefa ou categoria não encontrada

Status	Quando acontece
409	E-mail já cadastrado; categoria que ainda tem tarefas

Decida o que fazer pelo **status code**, e use `message` apenas para mostrar o texto ao usuário.

8. Testando sem o front

Teste primeiro no terminal ou em ferramentas como Postman, Insomnia ou Thunder Client. Se funcionar ali e falhar no front, o problema costuma ser CORS, header ou o corpo da requisição.

```
# 1. Login (copie o token da resposta)
curl -X POST http://localhost:8080/api/auth/login -H "Content-Type:
application/json" -d '{"email":"maria@email.com","password":"minhaSenha123"}'

# 2. Listar categorias (troque SEU_TOKEN)
curl http://localhost:8080/api/categories -H "Authorization: Bearer
SEU_TOKEN"
```

No navegador, a aba **Network** do DevTools (F12) mostra cada requisição com URL, status, headers e corpo da resposta. É a principal ferramenta para descobrir o que está acontecendo.